

MPStab: an hybrid stabilizers tensor-network quantum circuit simulator

Giulio Crognaletti,^{1,2,3} Mattia Robbiano,³ Michele Grossi,⁴ and Matteo Robbiati⁵

¹*Department of Physics, University of Trieste, Strada Costiera 11, 34151 Trieste, Italy*

²*Istituto Nazionale di Fisica Nucleare, Trieste Section, Via Valerio 2, 34127 Trieste, Italy*

³*Department of Physics, University of Helsinki, P.O. Box 43, FI-00014 Helsinki, Finland*

⁴*European Organization for Nuclear Research (CERN), Geneva, Switzerland*

⁵*Department of Microtechnology and Nanoscience,
Chalmers University of Technology, SE-412 96 Gothenburg, Sweden*

The development of techniques for simulating quantum systems using classical computers is a paramount task for two primary reasons: *i)* there exist configurations for which classical computers are remarkably effective and will continue to be so, and *ii)* exploring the limits of classical computation facilitates the identification of the regimes of competence for quantum computers. In this work, we present **mpstab**, a quantum circuit simulator based on a hybrid formalism combining stabilizers and tensor networks, recently introduced in [1]. We present the package, its core functionalities, and explore its performances in a few interesting simulation regimes.

CONTENTS

I. Introduction	1
II. Hybrid stabilizer-MPO	2
A. Circuit decomposition	2
B. Tensor network representation	3
C. Mixed picture evolution of HSMPO	3
III. The mpstab simulator	4
A. Basic usage and API	4
B. Flexibility and modularity	5
C. Integration with existing workflows	5
D. Computational complexity	6
1. Comparison with standard MPS	7
IV. Benchmarks	7
A. Benchmark setup	8
B. Numerical results	9
V. Conclusions	11
VI. Acknowledgments	12
References	12

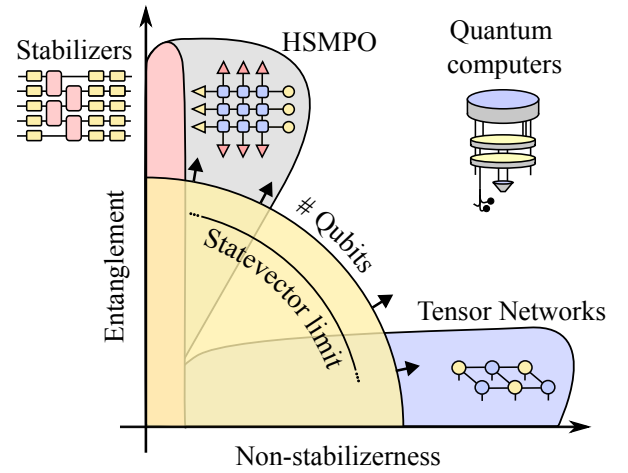


FIG. 1. A qualitative representation of classical simulator realms. Laying on the entanglement axis, stabilizers simulators allow representing pure-Clifford circuits at large scale. On the magic axis, tensor networks consent simulation of large systems if entanglement growth remains under control. Mixed approaches, like the hybrid stabilizer-matrix product operator (HSMPO) implemented here, push the boundaries of the classical simulators closer to the bijector of the plane, which corresponds to the quantum computers realm.

I. INTRODUCTION

Simulating quantum systems on classical computers is a fundamental challenge in quantum information science. The most straightforward approach, statevector simulation, represents the quantum state as a vector and applies gates sequentially in the Schrödinger picture. Several implementations are available, such as Qiskit [2], Cirq [3], Pennylane [4], Qulacs [5] and Qibo [6] among others. This method is simple and exact, making it ideal for validating small quantum algorithms and benchmarking. However, the state space dimension grows exponentially with system size, rendering this approach intractable beyond a few dozen qubits. Even with modern

improvements such as just-in-time compilation [7] or distributed computing, this exponential scaling remains an insurmountable bottleneck for simulating larger systems.

Therefore, researchers have developed structured algorithms that exploit specific properties of quantum circuits to extend the classical simulation boundary. These approaches succeed by restricting attention to special dynamical regimes where the full state space dimension is not needed. In practice, as sketched in Fig. 1, the difficulty of simulating many-body quantum dynamics stems from two competing sources of computational complexity: *entanglement* and *non-stabilizerness*. Quantum computers become necessary when both resources are extensively present, and classical simulators fail. In-

deed, on one hand, tensor networks and particularly matrix product states (MPS), excel at simulating systems with local interactions. They are especially powerful for short-time dynamics and ground state calculations of local Hamiltonians where entanglement remains bounded. However, in generic quantum evolution, rapid entanglement growth forces an exponential increase in the tensor-network bond dimension, leading to a breakdown of efficiency [8]. Various tensor network simulators have been introduced in the last years, such as Quimb [9], ITensor.jl [10], TenPy [11] and Quantum Tea [12].

On the other hand, stabilizer methods sit at the opposite extreme. They bypass the entanglement barrier, enabling exact polynomial-time simulation of highly entangling circuits [13] by tracking Pauli stabilizers with efficient tableau arithmetic. Yet, they lack universality, supporting only *Clifford* gates that map Pauli strings into other Pauli strings. Because non-Clifford gates map Pauli operators into complex superpositions, extending these stabilizer simulations to accommodate operations like T gates or arbitrary angle rotations generally incurs in an exponential cost. The most widely used package for pure Clifford simulations is Stim [14].

Several hybrid and specialized approaches have emerged to address broader dynamical regimes. *Pauli propagation* methods evolve observables (rather than the full state) and maintain sparsity through truncation, proving effective for noisy circuits and systems with arbitrary connectivity [15]. *Lie-algebraic simulations* [16] exploit circuits' underlying group structure for efficient evolution when the dynamical Lie algebra remains small. *Neural Quantum States* [17] leverage neural networks to parameterize volume-law entangled states inaccessible to tensor networks. Among them, the Hybrid Stabilizer-Matrix Product Operator (HSMPO) framework [1] was recently introduced to bridge tensor networks and stabilizer methods. Exploiting the Pauli-preserving property of Clifford conjugations we can push Clifford gates past non-Clifford rotations and represent the result as compact tensor networks with fixed bond dimension. This method achieves polynomial-time simulation for circuits dominated by Clifford gates yet interspersed with moderate non-Clifford rotations, that neither tensor networks nor stabilizers can efficiently handle alone.

It is in this context that we introduce `mpstab`, a Python library implementing the HSMPO formalism for hybrid quantum circuit simulation.

In the following, we first summarize the HSMPO formalism in Sec. II. Then, we introduce `mpstab` in Sec. III, describing its main utilities and showcasing usage examples alongside performance benchmarks against other state-of-the-art simulators.

🔓 Open-source package:

<https://github.com/MatteoRobbiati/mpstab>.

📖 Documentation:

<https://matteorobbiati.github.io/mpstab>.

II. HYBRID STABILIZER-MPO

Here we summarize the workings of HSMPO simulations, a technique introduced in Ref. [1] which attempts to bridge tensor networks and stabilizer methods, highlighting the role of both components into the algorithm.

A. Circuit decomposition

We start from the observation that any universal quantum circuit, like the example shown in Fig. 2, can be visualized as alternating Clifford unitary blocks and single-qubit rotations around Pauli axes:

$$U = C_0 R_1 C_1 R_2 C_2 \cdots R_{N_{\text{nc}}} C_{N_{\text{nc}}}, \quad (1)$$

where each C_j is a potentially *non-local* Clifford unitary and each magic gate $R_j = \exp(-i\theta_j \sigma_{\mu_j}/2)$ is a rotation by angle θ_j around axis $\mu_j \in \{X, Y, Z\}$ on a single qubit (or a *local* rotation). A second central observation is that

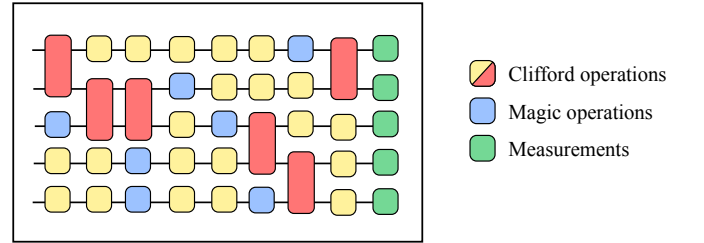


FIG. 2. Any quantum circuit can be written in terms of Clifford operations (red and yellow gates) alternated with local magic operations (blue gates).

conjugation by a non-local Clifford C will in general map a local Pauli P to a non-local Pauli $CPC^\dagger$. For example, consider a local rotation on qubit 2 of a 5-qubit system, generated by the tensor product

$$I \otimes I \otimes Z \otimes I \otimes I \equiv IIZII, \quad (2)$$

so that the rotation acts as

$$R = \exp\left[-i\frac{\theta}{2} IIZII\right]. \quad (3)$$

Applying a CNOT_{02} (control on qubit 0, target on qubit 2) followed by a Hadamard on qubits 0 and 2, the generator transforms under conjugation as

$$IIIZII \xrightarrow{\text{CNOT}_{02}} ZIZII \xrightarrow{H_0 \otimes H_2} XIXII, \quad (4)$$

where Z on the target backpropagates to the control qubit via $Z_t \rightarrow Z_c Z_t$, and $HZH^\dagger = X$ maps each remaining Z to X . The originally local rotation $\exp(-i\theta Z_2/2)$ is thus conjugated into the genuinely non-local two-body interaction $\exp(-i\theta X_0 \otimes X_2/2)$, which generates entanglement between qubits 0 and 2.

This motivates moving Clifford gates past the rotation operators via conjugation:

$$C_j R_j C_j^\dagger = \tilde{R}_j, \quad (5)$$

where $\tilde{R}_j = \exp(-i\theta_j P_j/2)$ and $P_j = C_j \sigma_{\mu_j} C_j^\dagger$ is now a (possibly non-local) Pauli string. By repeatedly applying this conjugation, any Clifford block can be “pushed through” the rotation to its right, absorbing the Pauli-string rotation without changing the unitary:

$$U = (C_1 R_1 C_1^\dagger)(C_1 C_2 R_2 C_2^\dagger C_1^\dagger) \cdots = \tilde{R}_1 \tilde{R}_2 \cdots \tilde{C}, \quad (6)$$

where $\tilde{R}_j$ are the dressed rotations and $\tilde{C}$ is the total residual Clifford evolution.

At this point it is worth pausing to understand *why* this rewriting is advantageous for classical simulation.

Indeed Eq. (6) cleanly separates two distinct roles that the Clifford layers are responsible for: *i*) spreading entanglement through the state *after* any magic rotation is applied, and *ii*) dressing the Pauli axes along which the magic rotations act on. The stabilizer MPO formalism takes advantage of this separation, addressing each point with the most suitable simulation technique.

In particular, role *i*) is handled by the stabilizer simulator, which tracks Clifford evolution in polynomial time in Heisenberg picture (see Sect. II C), while role *ii*) is handled analytically and exactly, by replacing the Pauli generator of each $\tilde{R}_j$. What remains to simulate is only the sequence $\tilde{R}_1 \tilde{R}_2 \cdots$, each of which, as we show in the next section, *always* admits a bond-dimension-2 MPO representation, and can therefore efficiently handled by a tensor network approach. In this way, the entanglement injected into the state by the full sequence $\tilde{R}_1 \cdots \tilde{R}_{N_{\text{nc}}}$, lacking most entangling gates, grows slower than conventional techniques, allowing the MPS simulation to remain accurate at fixed bond dimension χ for substantially longer times compared to direct application of U . This is well explained and documented in the original work of Mello et al. [1], where some dynamics examples are also showcased.

B. Tensor network representation

The dressed operators $\tilde{R}_j$ admit compact Matrix Product Operator (MPO) representations with bond dimension exactly two. Specifically, any Pauli-string rotation

$$\exp\left[-i\frac{\theta}{2}\Sigma_\gamma\right] = \cos(\theta/2)I - i\sin(\theta/2)\Sigma_\gamma, \quad (7)$$

where $\Sigma_\gamma = \sigma^{\gamma_1} \otimes \sigma^{\gamma_2} \otimes \cdots \otimes \sigma^{\gamma_L}$ is a tensor product of single-site Pauli operators, can be written as a bond-2 MPO. To see this explicitly, note that Eq. (7) is a linear combination of exactly two operators: the identity $I^{\otimes L}$, which is itself a product operator with bond dimension one, and the Pauli string Σ_γ , which is also a product

operator with bond dimension one. Their linear combination can therefore be written as a single MPO with bond dimension two, whose local tensors at each site l take the block-diagonal form

$$W_l = \begin{pmatrix} \phi_0 \sigma^0 & 0 \\ 0 & \phi_1 \sigma^\gamma \end{pmatrix}, \quad (8)$$

where $\phi_0 = \cos^{1/L}(\theta/2)$ and $\phi_1 = (-i\sin(\theta/2))^{1/L}$ are the site-local factors of the two coefficients, distributed evenly across all L sites, and $\sigma^0 \equiv I$ is the single-site identity. The full operator is recovered by contracting the auxiliary indices across all sites:

$$\tilde{R}_j = W_1 W_2 \cdots W_L, \quad (9)$$

where the matrix multiplication acts on the bond (auxiliary) indices, while the Pauli matrices act on the physical indices. The key observation is that this bond dimension of two is an *exact* representation, and it is entirely independent of how many qubits the string Σ_γ acts on non-trivially. A weight- L Pauli string rotation, spanning the entire system, costs no more to represent as an MPO than a single-qubit rotation. It is precisely this property that makes the stabilizer MPO decomposition efficient: regardless of how non-local the dressed Pauli string P_j becomes after Clifford conjugation, each $\tilde{R}_j$ retains a bond-2 MPO structure, and the total bond dimension of the evolved state grows only through the sequential application of these rank-2 layers, at a rate controlled by the angles $\{\theta_j\}$.

C. Mixed picture evolution of HSMPO

In this context, we are interested in computing expectation values of linear combinations of Pauli observables like

$$O = \sum_i c_i P_i, \quad (10)$$

where P_i and c_i are a Pauli strings involving any number of qubits of our system and c_i is their coefficients respectively. If we consider the circuit decomposition introduced in Eq. (6), the expectation value of O over the state prepared by U can be computed as:

$$\begin{aligned} \langle O \rangle &= \langle \psi | U^\dagger O U | \psi \rangle = \\ &= \langle \psi | \tilde{R}_1^\dagger \cdots \tilde{R}_{N_{\text{nc}}}^\dagger \tilde{C}^\dagger O \tilde{C} \tilde{R}_1 \cdots \tilde{R}_{N_{\text{nc}}} | \psi \rangle. \end{aligned} \quad (11)$$

HSMPO adopts a mixed *forward-backward* picture for the time evolution implementation, motivated by the decomposition introduced in Eq. (6). On one hand, Clifford operations are applied in the Heisenberg picture, acting on observables. This is why we often refer to these parts of the evolution as *backpropagations*; an initial observable O is evolved into a new observable O' through the action of N_C Clifford operations via tableau arithmetic,

according to the stabilizer formalism. This operation has a computational cost which scales as $O(N_C L)$, where L is the number of qubits [13, 18].

On the other hand, non-Clifford operations are implemented in Schrödinger picture, namely acting on the state with a *forward* evolution. In this non-Clifford evolution, each gate $\tilde{R}_j$ is represented as an MPO, as given in Eq. (7), and contracted directly with the MPS encoding the initial state. After each contraction, the bond dimension of the resulting state is truncated to remain below a prescribed maximum value χ . Starting from an initial state $|\psi\rangle$, the gates $\tilde{R}_j$ are applied iteratively, with a compression step following each contraction. This approach leads to a computational complexity which scales with the maximum bond dimension and the number of magic operations N_{nc} as $O(\chi^3 N_{\text{nc}})$. After the evolution steps, the Pauli string corresponding to the evolved operator O' is applied to the evolved MPS $|\psi\rangle$, and the expectation value is computed as a tensor network contraction:

$$\begin{aligned} \langle O \rangle &= \langle \psi | (\tilde{R}_1^\dagger \cdots \tilde{R}_{N_{\text{nc}}}^\dagger) (\tilde{C}^\dagger O \tilde{C}) (\tilde{R}_1 \cdots \tilde{R}_{N_{\text{nc}}}) | \psi \rangle \\ &= \langle \psi' | U^\dagger O' U | \psi' \rangle. \end{aligned} \quad (12)$$

It is worth noting that the two components of this scheme carry different levels of exactness. The stabilizer evolution is exact, introducing no approximation error. The tensor network evolution, by contrast, is approximate: truncating the bond dimension limits the maximum entanglement the simulation can faithfully represent. This hybrid strategy is not unique to the present work; related approaches combining exact Clifford treatment with approximate tensor network methods have been explored in the literature [19].

III. THE MPSTAB SIMULATOR

To turn the HSMPO framework into a practical tool, we developed **mpstab**, an open-source Python package that implements hybrid stabilizer-tensor network simulation. Find the source here:

 <https://github.com/MatteoRobbiati/mpstab>.

The library is designed to integrate seamlessly with **Qibo** [6], a high-level quantum computing framework, allowing users to express quantum circuits in familiar syntax while leveraging the HSMPO method to simulate more complicated circuits.

As summarized in Fig. 3, the package is structured into three main modules: *i*) the engines, deputed to the execution of the main operations described in the previous sections, *ii*) the evolvers, namely our main interface, where the HSMPO formalism takes the form of a class and can be used to perform simulations. Finally, *iii*) the models module contains a few built-in quantum circuit ansatze and some higher-level functionalities, such as experimental error mitigation methods relying

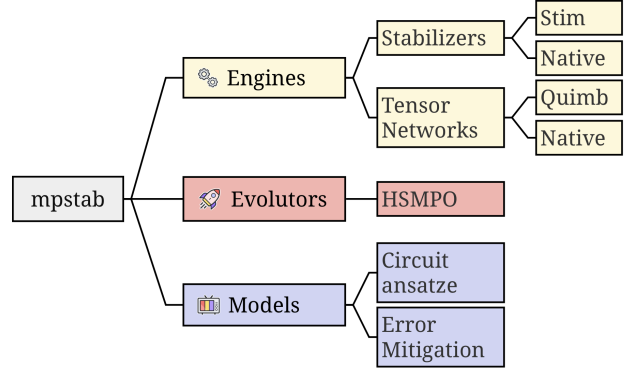


FIG. 3. Schematic representation of **mpstab** 0.1.0.

on the HSMPO formalism or some relevant metrics, such as exact or stochastic estimate of the Stabilizer Rényi Entropy [20]. These components represent the back-bone structure of **mpstab** and will be expanded in future releases including new features, such as noise simulation.

At the user-facing level sits the **HSMPO** class, which orchestrates the entire workflow: it partitions an input circuit into Clifford and non-Clifford blocks, conjugates magic rotations with preceding Clifford gates to ensure Pauli structure is preserved, manages the MPS state evolution, applies truncation, and finally evaluates expectation values with automatic fidelity tracking. This abstraction insulates users from low-level technical details while maintaining the flexibility to swap engines and control key parameters such as the maximum bond dimension.

We provide our own tensor network and stabilizer simulators implementations, together with a Quimb [9] engine (for TN simulations) and a Stim [14] one (for Stabilizers).

A. Basic usage and API

As anticipated, our main focus is providing a simple framework to simulate a quantum circuit in HSMPO mode. For this reason, we delegate the quantum circuit and observables syntaxes to Qibo [6].

After defining a quantum circuit using one of our built-in models or using the Qibo interface, one instantiates an **HSMPO** object as shown in the following code snippet

```

1 from qibo import Circuit, gates
2 from mpstab.evolutors.hsmpto import HSMPO
3 from mpstab.models.ansatze import
   HardwareEfficient
4
5 # Create a parameterized circuit using a
   built in model

```

```

6 circuit = HardwareEfficient(nqubits=5,
    nlayers=3)
7
8 # or the Qibo syntax
9 circuit = Circuit(5)
10 for q in range(5):
11     circuit.add(gates.RY(q=q, theta=0.34))
12 circuit.add(CZ(0, 3))
13 circuit.add(CZ(4, 1))
14
15 # Initialize simulator choosing the bond
    dimension
16 sim = HSMPO(circuit, max_bond_dimension=64)

```

When instantiating an HSMPO object, one can set the maximum bond dimension of the MPS representation.

The evolution of the MPS through the series of the dressed rotations $\tilde{R}$ is computed once, and then cached to speed up the cost of computing multiple expectation values, as it happens when we consider an observable composed of many terms. An example of expectation value calculation is shown in the following code snippet

```

1 from qibo.symbols import X
2 from qibo.hamiltonians import
    SymbolicHamiltonian
3
4 # Define an arbitrary symbolic hamiltonian
5 ham_form = 0
6 for q in range(5):
7     ham_form += X(q%5) * X((q+1)%5)
8 hamiltonian = SymbolicHamiltonian(ham_form)
9
10 # Compute the expectation value using HSMPO
11 sim.expectation(observable=hamiltonian)
12
13 # Expectation of the same observable using
    less magic circuit than the original
14 sim.expectation_from_partition(
15     observable=hamiltonian,
16     replacement_probability=0.7,
17     replacement_method="random",
18 )

```

In the last block of code, we show a particular feature of `mpstab`, namely the expectation value computed over a processed version of the original circuit. If this method is called, `HSMPO` takes the original quantum circuit and replaces its magic gates (local rotations around general angles) with Clifford operations with an arbitrary chosen probability. This is done by setting the rotation angle to be a multiple of $\pi/2$. This feature is particularly useful when implementing data-driven error mitigation techniques or when exploring the simulator performances in various regimes of magic-to-stabilizerness ratio.

B. Flexibility and modularity

A key design principle of `mpstab` is modularity. Users can select different stabilizer and tensor network engines independently. For instance, one might use `StimEngine` for fast Pauli propagation paired with `QuimbEngine` configured on its Jax backend to enable automatic differentiation in case, for example, an HSMPO model is requested within a variational optimization pipeline. We propose this flexibility in order to consent the implementation of dedicated engines, which can be effortlessly integrated in our workflow. One can thus choose among the available engines and set them as shown in the following code snippet.

```

1 from mpstab.engines import StimEngine,
    QuimbEngine
2
3 # Choose engines independently
4 sim.set_engines(
5     stab_engine=StimEngine(),
6     tn_engine=QuimbEngine(backend="jax")
7 )

```

So far, the modular structure of `mpstab` has been following a specific HSMPO structure, having stabilizer engines executing the backpropagation of Pauli strings and tensor network engines implementing the tensor network representation of the remaining part of the evolution.

However, although this was the first mechanism implemented, we see significant room for exploration in terms of the engines used. For example, one could derive the analytical form of non-local rotations using a stabilizer engine and then attempt to synthesize a circuit in the style of digital quantum computing starting from there. Alternatively, one could consider using more complex tensor network ansatzes or alternative hybrid approaches such as Pauli Propagation [15]. A similar solution has indeed been implemented in [21].

C. Integration with existing workflows

To enable a further modular usage of `mpstab`, we enable it to be registered as a Qibo backend. This can be easily done by using the dedicated backends API implemented in Qibo. We provide a simple working example in our documentation at [22].

The ability to use the HSMPO formalism whilst leveraging all the tools of a rich and well-established framework such as Qibo significantly speeds up the exploration of new solutions, such as the use of HSMPO in variational contexts like those for which QiboML was introduced [23].

More remarkably, Qibo offers a simple interface to real quantum devices, and its backends can be easily constructed and used in synergy to propose hybrid quantum-classical pipelines.

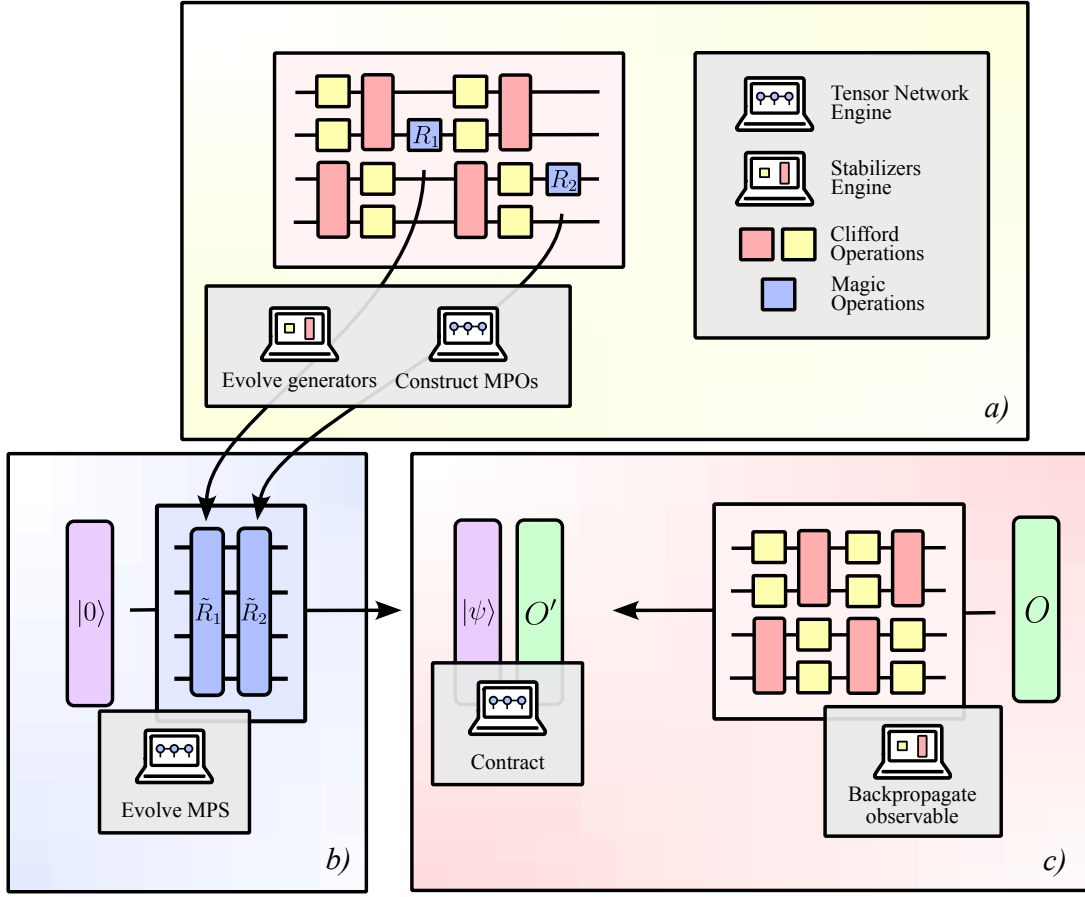


FIG. 4. `mpstab`'s operational workflow when computing the expectation value of an observable O over the state represented by an HSMPO. A tensor network and a stabilizers engines cooperate to prepare the evolved MPS and use it to perform the contraction with the given observable.

Modularity plays a crucial role in nowadays quantum computing applications, where quantum and classical components act as hardware accelerators within hybrid contexts.

D. Computational complexity

The computational cost of computing an expectation value with `mpstab` is determined by the interplay between tensor network and stabilizer operational engines. To give a graphical intuition of the computational workflow, we support the next discussion with the schematic representation shown in Fig. 4.

The whole computational workflow can be divided into three steps:

- a) **Operation setup:** identification of the Clifford blocks and magic rotations, and construction of the dressed rotations.
- b) **MPS evolution:** an initial state, represented as an MPS, is evolved through the layer of dressed rotations.

- c) **Expectation value evaluation:** the target observable is evolved under the cumulative Clifford layer and contracted with the MPS obtained at step b.

Let L be the number of qubits and N be the total number of gates, where $N = N_C + N_{nC}$ represents the sum of Clifford and non-Clifford operations.

a. Operation setup. This first operation consists in locating the position of magic gates in the queue of operations composing the quantum circuit, so that we can identify the Clifford blocks and the local rotations mentioned in Eq. (1). This requires a single pass over the gate list, scaling as $O(N)$. For every magic gate R_i in the decomposition of Eq. (1), we must evolve the rotation axis through the non-local Clifford circuit built from all Clifford gates preceding the target local rotation R_i in the original circuit. In the symplectic representation of the stabilizer formalism, each elementary 1- or 2-qubit Clifford gate updates only $O(1)$ entries of the Pauli string encoding the rotation axis. Consequently, the computational cost of preparing the generator of the dressed rotation $\tilde{R}_i$ scales as $O(N_{C,i})$, where $N_{C,i}$ is the number

of Clifford gates preceding the i -th rotation.

Considering the extreme case where each rotation is dressed under the action of all the Clifford gates populating the circuit, the computational cost of this step scales as $O(N_{\text{nC}} N_{\text{C}})$.

b. MPS evolution. Once the generators of the dressed rotations are constructed, these non-local operations are applied as MPOs to the initial state, which is written in the form of an MPS. Contracting an MPO tensor (with bond dimension $D = 2$, as shown in Sec. II B) with an MPS tensor of bond dimension χ scales as $O(d^2 D^2 L \chi^2) = O(16 L \chi^2)$ for physical dimension $d = 2$. Following the contraction, an SVD-based compression is performed to keep the bond dimension bounded by χ ; the SVD of each local matrix of size $(d\chi D) \times (\chi D)$ costs $O(dD^3 \chi^3)$, giving a total $O(dD^3 L \chi^3) = O(16 L \chi^3)$. The full evolution of a magic block is therefore dominated by the SVD step and scales as $O(16 L \chi^3)$.

Referring back to the two sub-routines mentioned at the beginning of this section, the MPS evolution is computed in `mpstab` through steps *a* and *b*, while the expectation value calculation corresponds to step *c*.

c. Expectation value of a target observable. The evaluation of the expectation value of a target observable O requires backpropagating the observable through the full Clifford circuit, obtained by concatenating all the Clifford blocks identified at step *a*. By the same argument used for the dressing, this operation costs $O(N_{\text{C}})$ per Pauli term in the observable; assuming an observable with $O(L)$ Pauli terms (as is typical for k -local Hamiltonians) yields a total cost of $O(L N_{\text{C}})$. The resulting evolved Pauli string, represented as a product operator, is then contracted with the evolved MPS $|\psi'\rangle$. This final contraction between the Pauli string and the state scales as $O(d^2 L \chi^2) = O(4 L \chi^2)$, ensuring that the measurement phase remains computationally efficient.

1. Comparison with standard MPS

In standard Matrix Product State (MPS) simulation, gates are applied sequentially to the state representation. As discussed in [24], single-qubit gates can be applied to an MPS state with cost $O(\chi^2)$, adjacent two-qubit gates with cost $O(\chi^3)$, and non-adjacent two-qubit gates acting across spatial distance r require swap sequences costing $O(r\chi^3)$. For a circuit of N gates with maximum interaction range ℓ , the total upper-bound complexity is $O(N \ell \chi^3)$, with a final contraction of the observable with the state adding $O(d^2 L \chi^2) = O(4 L \chi^2)$ for physical dimension $d = 2$.

Letting $N = N_{\text{C}} + N_{\text{nC}}$ denote the total gate count, split into N_{C} Clifford and N_{nC} magic gates, and assuming worst-case non-locality $\ell = L$, the standard MPS cost

reads:

$$C_{\text{mps}} = (N_{\text{C}} + N_{\text{nC}}) L \chi_{\text{mps}}^3 + 4 L \chi_{\text{mps}}^2, \quad (13)$$

where the *mps* label on the bond dimension emphasises that MPS and HSMPO representations may require different bond dimensions to achieve the same accuracy in the comparison with `mpstab`.

Aggregating the costs derived in Sec. III D, setup, stabilizer backpropagation of the observable, dressing of the magic rotations, MPS evolution through the dressed rotations, and final measurement, the `mpstab` cost is:

$$C_{\text{mpstab}} = N + L N_{\text{C}} + N_{\text{nC}} N_{\text{C}} + 16 N_{\text{nC}} L \chi_{\text{mpstab}}^3 + 4 L \chi_{\text{mpstab}}^2, \quad (14)$$

where the $L N_{\text{C}}$ term accounts for the backpropagation of an observable with $O(L)$ Pauli terms through all Clifford operations, and $N_{\text{nC}} N_{\text{C}}$ collects the cost of dressing each of the N_{nC} magic rotations under all preceding Clifford gates.

We analyse the ratio $C_{\text{mps}}/C_{\text{mpstab}}$ as the primary performance metric. Figure 5 maps this ratio across the circuit-composition space for six pairs of maximum bond dimensions $(\chi_{\text{mps}}, \chi_{\text{mpstab}})$. The three values $\chi_{\text{mpstab}} \in \{8, 32, 256\}$ span from strongly compressed to weakly compressed hybrid representations, while the two ratios $\chi_{\text{mps}}/\chi_{\text{mpstab}} \in \{1.1, 2.5\}$ encode the bond-dimension saving that `mpstab` achieves at fixed fidelity; a larger ratio corresponds to a stronger fidelity advantage of the hybrid method.

Two regimes are visible. For small and moderate χ_{mpstab} (left and middle columns), the iso-cost contour has an arched shape: `mpstab` is cheaper in a finite window of N_{C} that closes at very large Clifford counts, where the $N_{\text{nC}} N_{\text{C}}$ contribution to the `mpstab` cost eventually dominates. For large χ_{mpstab} (right column) this term is subleading, and the boundary becomes a straight diagonal in log-log scale, set primarily by the cubic bond-dimension gap $(\chi_{\text{mps}}/\chi_{\text{mpstab}})^3$.

Increasing the ratio $\chi_{\text{mps}}/\chi_{\text{mpstab}}$ (top to bottom row) systematically expands the `mpstab`-favourable region, reflecting the cubic scaling with bond dimension of the standard method. Overall, the `mpstab` advantage is most pronounced at low N_{nC} and high N_{C} , where the stabilizer formalism absorbs the bulk of the entanglement structure and the non-Clifford overhead remains modest.

IV. BENCHMARKS

As widely discussed, HSMPO is introduced as a tool to simulate longer dynamics and more entangled operations. While this is true on a theoretical perspective, implementing this in practice introduces a series of overheads and complexities that makes non-trivial the task of identifying when our simulator is supposed to be the to-go choice among the rich plethora of alternatives.

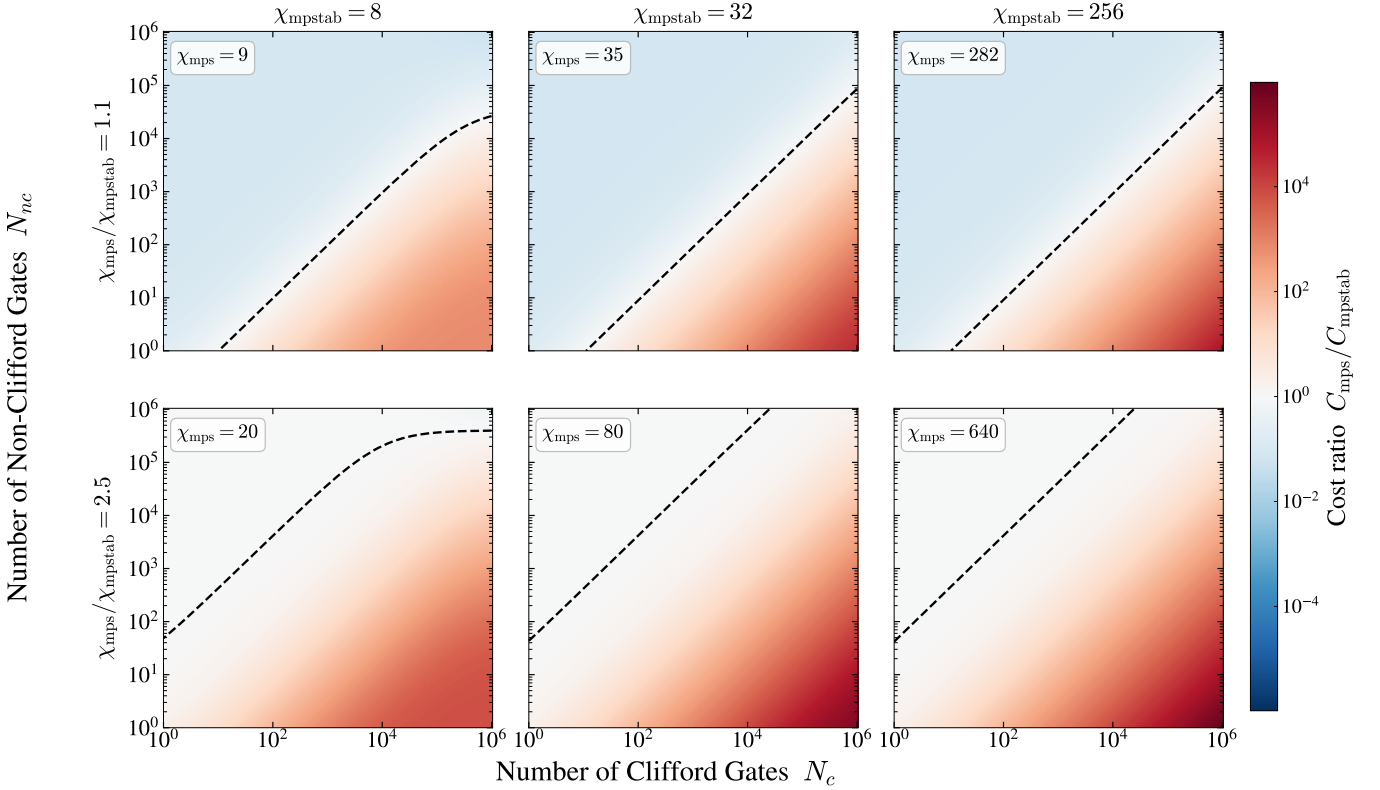


FIG. 5. Cost ratio $C_{\text{mps}}/C_{\text{mpstab}}$ as a function of Clifford (N_c) and non-Clifford (N_{nc}) gate counts. Red regions indicate where **mpstab** is cheaper than standard MPS; blue regions indicate the opposite. The dashed curve marks the iso-cost boundary $C_{\text{mps}} = C_{\text{mpstab}}$. Columns fix the **mpstab** bond dimension $\chi_{\text{mpstab}} \in \{8, 32, 256\}$, spanning strongly- to weakly-compressed hybrid representations; rows fix the ratio $\chi_{\text{mps}}/\chi_{\text{mpstab}} \in \{1.1, 2.5\}$ between the two methods, corresponding respectively to a weak and a strong fidelity advantage of the hybrid method. The leftmost column exhibits the arched regime, where the stabilizer overhead $N_{nc} N_c$ saturates the boundary at high N_c ; the middle and right columns exhibit the diagonal regime, where the boundary is set entirely by the cubic bond-dimension gap. System size is $L = 50$ with worst-case interaction range $\ell = L$.

With this in mind, and considering the task of computing expectation values, we want to answer here the question: *for which kind of circuits **mpstab** is the best available simulator?*

It is natural for us to structure these benchmarks *against* a pure tensor network simulator. This allows the analysis of **mpstab**'s performances to be the fairest as possible considering two aspects: *i)* the alternative method (TN) doesn't present any overhead and *ii)* using Quimb, we can easily force the pure Tensor Network simulation and the **mpstab**'s tensor network engine to follow the same configuration: contractions optimizer, backend, etc.

A. Benchmark setup

The two main characteristics impacting on the analysis of the performance are the amount of *entanglement* and *magic* in the circuit. We therefore design an *ad hoc* ansatz that provides precise, independent control over these two resources, and complement it

with a metrology protocol that certifies the accuracy of **mpstab** without ever needing access to the exact state.

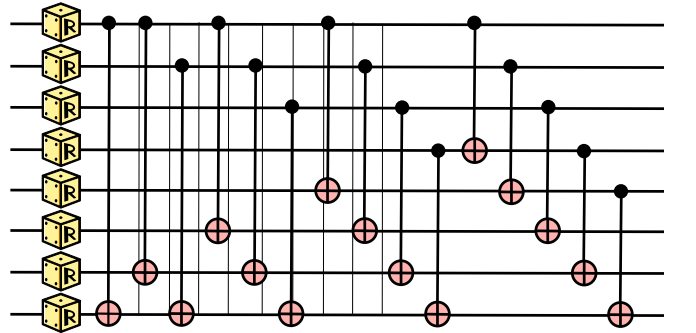


FIG. 6. One layer of the quantum circuit used for benchmarking. First we have a layer of L single qubit rotations, a fraction m of which, will be Clifford, the rest will be random. The interaction part consists of a CNOT ladder containing all possible CNOT gates from interaction length $L - 1$ down to $k = 5$.

a. Ansatz. Each layer of the benchmark ansatz, illustrated in Fig. 6, consists of L single-qubit rotations followed by a CNOT ladder of tunable depth. The rotations are drawn so that a fraction β of them is non-Clifford, with random rotation angles, and the remaining $1 - \beta$ is Clifford, with angles multiple of $\pi/2$, so that the magic content of the layer can be dialed continuously. The interaction block is a ladder of CNOT gates spanning interaction lengths from $L - 1$ down to a cutoff k , yielding $k(k - 1)/2$ entangling Clifford gates per layer; smaller k means a deeper ladder and correspondingly more entanglement injected. Under this construction, the magic-to-Clifford ratio per layer reads

$$\frac{N_{\text{nC}}}{N_{\text{C}}} = \frac{L\beta}{L(1 - \beta) + k(k - 1)/2}, \quad (15)$$

so that the hyperparameters (β, k) span the two axes of the entanglement versus magic diagram of Fig. 1 in a controlled way. Throughout the benchmarks we tune the magic content using the quantity β , which we dub magic retention.

b. Certifying accuracy via a fidelity lower bound. Since we operate deep in the many-qubit regime, exact-state fidelities are inaccessible and we must rely on an intrinsic certificate of approximation quality. **mpstab** tracks a fidelity lower bound throughout the simulation: at each MPS compression step, truncating the singular values below the bond dimension threshold discards a portion of the state norm. Since unitary evolution preserves the norm exactly, any deviation of the final MPS norm from unity is entirely attributable to truncation. The fidelity lower bound is therefore

$$\mathcal{F}_{\text{lb}} = \frac{\|\psi_{\text{trunc}}\|^2}{\|\psi_0\|^2} = \|\psi_{\text{trunc}}\|^2, \quad (16)$$

which provides a cheap and rigorous certificate that requires no access to the exact state. In the following, whenever we refer to “fidelity” we mean this lower bound; the same quantity is also tracked for the pure TN baseline for a fair comparison.

c. Protocol and parameter grid. The **mpstab** package is focused on expectation value calculation. The inputs for a single test are the observable to be measured, the quantum circuit, the magic retention β , and the maximum bond dimension χ_{max} . The outputs are the expectation value, the fidelity lower bound of Eq. (16), and the execution time. For each configuration, both **mpstab** and Quimb are executed under identical conditions: the pure Quimb tensor network simulation and the tensor network component of **mpstab** share the same contraction algorithms and optimization settings, so that any observed difference in cost or accuracy is attributable to the hybrid formalism and not to implementation-level asymmetries. To collect statistics, each configuration is repeated $M = 20$ times with independent random initializations

of the non-Clifford rotations. The hyperparameter values scanned in this study are collected in Table I. Results are reproducible using the **mssim** package; instructions are available in the **mpstab** repository.

Parameter	Values
Number of qubits L	20, 35, 50, 65, 80
Circuit depth D	1, 2, 3, 4, 5
Bond dimension χ_{max}	2, 4, 8, 16, 32
Magic retention β	0, 0.2, 0.4, 0.6, 0.8, 1.0

TABLE I. Hyperparameter values scanned in the benchmark simulations. Each configuration is repeated over $M = 20$ random initializations of the non-Clifford rotations.

B. Numerical results

We now walk through the three complementary views on **mpstab**’s performance provided by our benchmarks. The narrative follows the logic set up in Sec. IIID: we first confront the analytical cost model against wall-clock measurements to validate it, then dissect the two sources of **mpstab**’s advantage separately, namely its higher fidelity at fixed bond dimension (Fig. 8) and its wall-clock scaling with system size (Fig. 9).

a. Validating the cost model. Figure 7 shows the wall-clock execution time of **mpstab** and pure Quimb as a function of the single-qubit magic percentage, for two representative system sizes $L \in \{35, 80\}$ at fixed depth $D = 1$. As the magic percentage increases from 0 to 100%, **mpstab** moves from a fully stabilizer-tractable regime, where it exploits polynomial-time Clifford propagation, to a regime dominated by dressed non-Clifford rotations that must all be applied as MPOs to the MPS. Correspondingly, its execution time grows monotonically with the magic fraction, while pure Quimb, which is insensitive to the Clifford versus non-Clifford distinction, shows an essentially flat cost across the whole range.

The two curves cross at a well-defined magic fraction, marked in the figure by a vertical line at the value predicted by setting $C_{\text{mps}}/C_{\text{mpstab}} = 1$ in the cost model of Sec. IIID 1, Eqs. (13) and (14). The predicted crossover falls inside the interval where the measured curves actually cross, both at $L = 35$ and at $L = 80$, confirming that the cost model of Sec. IIID correctly predicts the order of magnitude of the crossover point, and therefore captures the leading-order scaling of the real implementation. This validation is important because it means the phase map of Fig. 5, drawn purely from the analytical model, can be trusted as a practical guide to when **mpstab** is expected to outperform pure MPS.

b. Fidelity advantage at fixed bond dimension. Once the cost model is validated, it is worth asking why **mpstab**

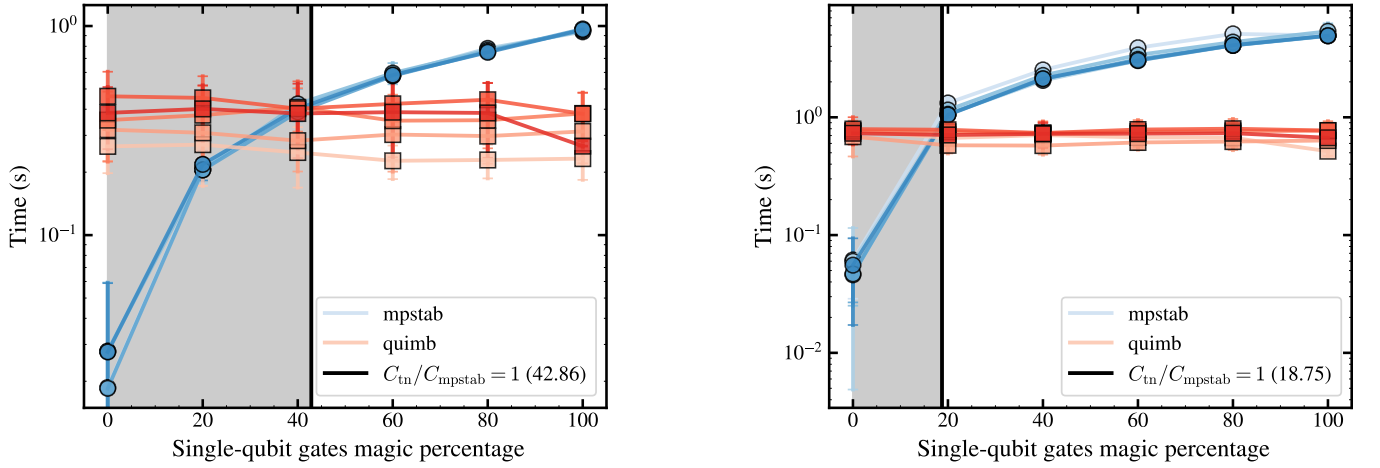


FIG. 7. Execution time as a function of magic fraction with a single circuit layer and varying bond dimensions. Blue and red lines correspond to **mpstab** and pure tensor network, respectively. The vertical line marks the predicted crossover point where computational costs are equivalent. *Left*: $L = 35$ qubits. *Right*: $L = 80$ qubits.

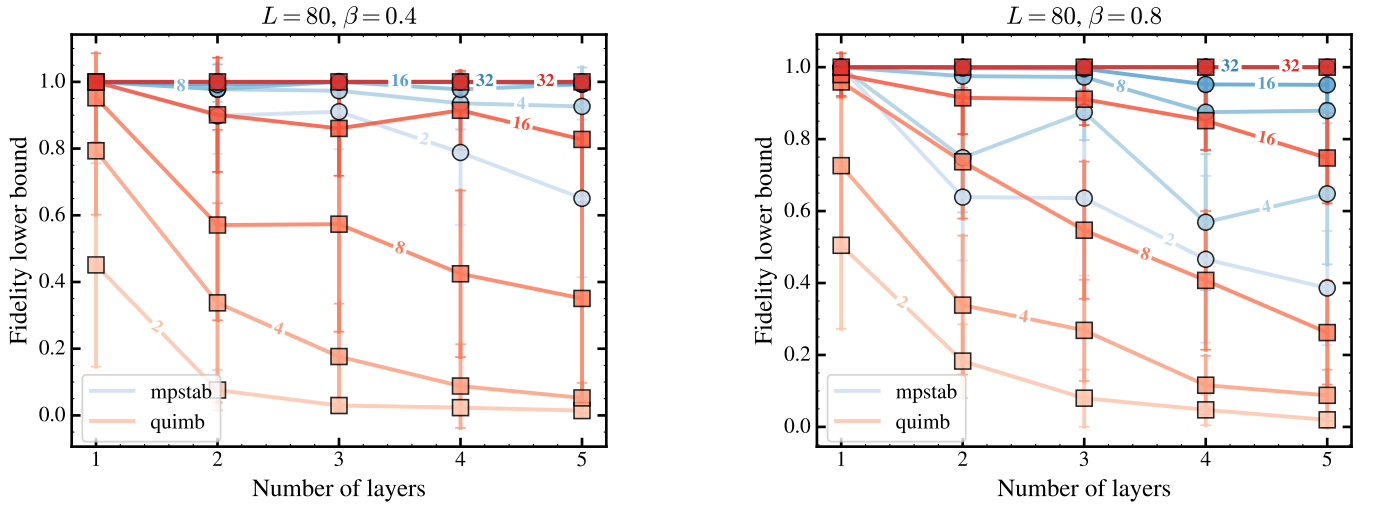


FIG. 8. Fidelity lower bound of **mpstab** expectation value computations for 80-qubit circuits as a function of circuit depth. Blue and red lines correspond to **mpstab** and pure tensor network, respectively. Different shades correspond to different values of $\chi_{\max}$, reported as labels on the lines. Markers and error bars show median and MAD of fidelity values across $M = 20$ unitaries. *Left*: $\beta = 0.4$, moderate magic. *Right*: $\beta = 0.8$, high magic.

can be cheaper than a pure tensor network at all, given that the underlying tensor network engine is the same. The answer, anticipated in Sec. II, is that Clifford conjugation absorbs a substantial portion of the entanglement structure of the circuit into the stabilizer formalism, so the MPS component of **mpstab** has to represent only the entanglement generated by the dressed non-Clifford rotations. At fixed bond dimension, this translates directly into a higher accuracy.

Figure 8 makes this concrete for 80-qubit circuits, plotting the fidelity lower bound of Eq. (16) as a function of circuit depth, at multiple values of $\chi_{\max}$. At fixed $\chi_{\max}$, **mpstab** (blue) is systematically above pure Quimb (red), and the gap widens with depth: as the circuit accumulates Clifford entanglement, the pure TN

representation degrades quickly while **mpstab** absorbs it exactly in the stabilizer component. The advantage is most pronounced at moderate magic ($\beta = 0.4$, left panel) where the stabilizer component contains a substantial fraction of the dynamics, but persists even at high magic content ($\beta = 0.8$, right panel), where **mpstab** still maintains significantly higher fidelity than the TN baseline at the same $\chi_{\max}$. This behavior is consistent with the picture underpinning the cost analysis of Sec. II D 1: reaching a target fidelity with **mpstab** requires a strictly smaller bond dimension than with pure MPS, i.e., $\chi_{\text{mps}} > \chi_{\text{mpstab}}$, which is precisely the regime in which the phase map of Fig. 5 favours the hybrid method.

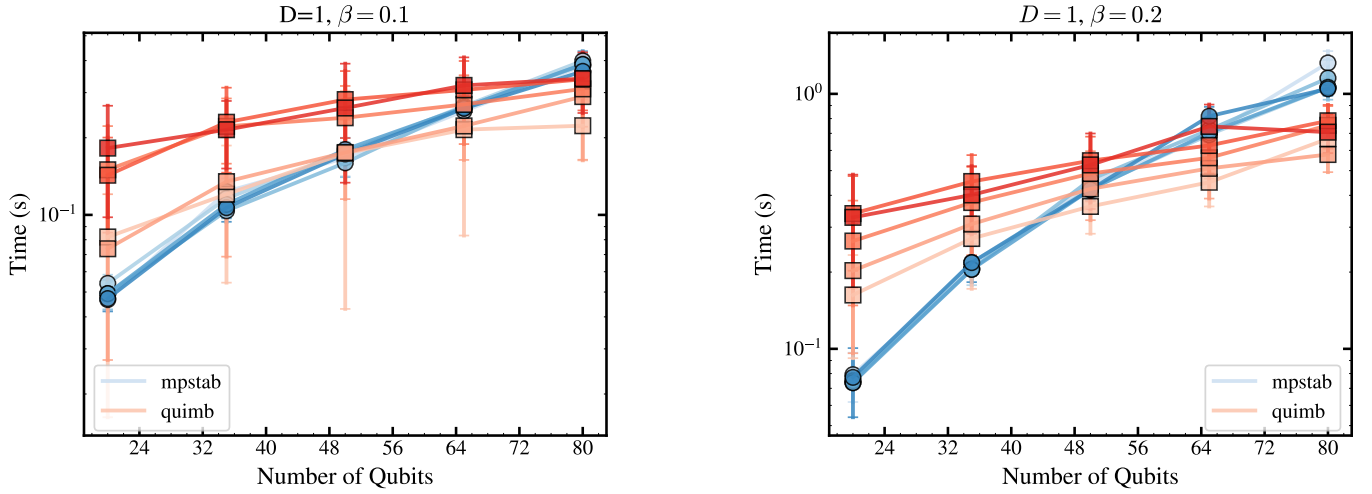


FIG. 9. Execution time as a function of number of qubits with a single circuit layer and varying bond dimensions. Blue and red lines correspond to `mpstab` and pure tensor network, respectively. Left: $\beta = 0.1$. Right: $\beta = 0.2$.

c. Wall-clock cost and overhead across system sizes.

The complementary question is how the two methods scale with system size in real wall-clock time. Figure 9 shows the execution time as a function of the number of qubits, at two fixed magic retention values and single-layer depth. Both curves grow monotonically with system size L , with the `mpstab` wall-clock following a steeper path. At low magic content ($\beta = 0.1$, left panel) `mpstab` performs well for a larger number of qubits compared to case of high magic content ($\beta = 0.2$, right panel), where advantage is confined to smaller systems. This result highlights once again that the setting most favorable to `mpstab` is that of low N_{NC} . Indeed the number of magic gates $N_{\text{NC}} \propto \beta L$ is by construction proportional to the system size in this example, meaning that for fixed β , an increase in the system size also implies an increase in N_{NC} , and thus the competitiveness of `mpstab` is reduced in accordance with the cost model of Sec. III D 1. Tuning the scaling of β with increasing L is therefore crucial to maintain the advantage at scale.

d. Answer to the guiding question.

Combining the three views, we can now return to the question that opened this section. `mpstab` is the preferred simulator whenever (i) the Clifford content is dominating, so that the stabilizer formalism absorbs a substantial share of the entanglement and (ii) the target fidelity requires a bond dimension high enough that the cubic-in- χ MPS cost dominates. This is precisely the region identified as `mpstab`-favourable in the analytical phase map of Fig. 5, low N_{NC} and high N_{C} , and it corresponds, physically, to Clifford-dominated circuits with moderate but non-trivial non-Clifford injection. Outside this window, that is, very low Clifford content, very small systems, or bond dimensions small enough that the tensor-network cost is not yet the bottleneck, the pure TN baseline remains a better choice.

V. CONCLUSIONS

We have presented `mpstab`, an open-source Python library implementing the hybrid stabilizer-MPO framework of Ref. [1]. By treating Clifford gates exactly and encoding non-Clifford rotations as bond-2 MPO layers, HSMPO extends the reach of classical simulation beyond what tensor networks and stabilizer methods can achieve independently, and `mpstab` turns this formalism into a practical tool.

Two design choices make `mpstab` convenient for daily use. Its modular architecture cleanly separates the stabilizer and tensor network engines, so either component can be swapped transparently, for instance with Stim or Quimb. Integration with Qibo places `mpstab` inside a broader ecosystem spanning circuit definition, variational optimization, and execution on real quantum hardware.

Our benchmarks locate the regime where the method shines: circuits with moderate magic and substantial Clifford content, where `mpstab` reaches higher fidelity than pure tensor networks at the same bond dimension, translating into real savings once the comparison is drawn at matched accuracy. The analytical cost model of Sec. III D correctly predicts where the crossover happens, giving a concrete map of the parameter space in which `mpstab` is the tool of choice.

Several directions remain open. On the algorithmic side, the Clifford blocks that precede a magic rotation shape the locality of the resulting dressed generator, and hence the bond-dimension growth of the MPS evolution; conversely, the Clifford operations sitting after all magic rotations dress the final observable, potentially turning a simple Pauli string into a complex sum of terms. A systematic study of how different Clifford substructures propagate axes and observables would help identify topologies that keep both objects as compact as possible.

On the software side, the modularity of `mpstab` in-

vites new engines: Pauli propagation for the observable side, richer tensor network ansatzes for the state side, or symbolic Clifford decomposers returning the dressed generators in analytical form. Each of these would probe a different corner of the entanglement versus magic diagram of Fig. 1.

Finally, applications where circuits are structurally low in magic map naturally onto `mpstab`. Learning-based error mitigation is a prime example: techniques like Clifford Data Regression train on surrogate circuits obtained by replacing most non-Clifford gates with Clifford ones, and are typically bottlenecked by the cost of generating those surrogates with exact simulators. `mpstab` removes this bottleneck, allowing a controlled amount of magic to be retained while scaling to system sizes well beyond exact simulation, and its replacement functionalities are

designed precisely for this use case. More broadly, we expect `mpstab` to be a useful component in any hybrid quantum-classical pipeline where the classical side must simulate near-Clifford surrogates of quantum circuits.

VI. ACKNOWLEDGMENTS

We thank M. Collura for useful discussions. Mattia Robbiano and Giulio Crognaletti gratefully acknowledge the financial support of the Finnish Foundation for Technology Promotion. Giulio Crognaletti also acknowledges financial support from University of Trieste and INFN. Michele Grossi is supported by CERN through the CERN Quantum Technology Initiative.

-
- [1] A. F. Mello, A. Santini, and M. Collura, *Physical Review Letters* **133**, 10.1103/physrevlett.133.150604 (2024).
 - [2] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, *Quantum computing with Qiskit* (2024), [arXiv:2405.08810](https://arxiv.org/abs/2405.08810) [quant-ph].
 - [3] C. Developers, *Cirq* (2025).
 - [4] V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, S. Ahmed, V. Ajith, M. S. Alam, G. Alonso-Linaje, B. Akash-Narayanan, A. Asadi, J. M. Arrazola, U. Azad, S. Banning, C. Blank, T. R. Bromley, B. A. Cordier, J. Ceroni, A. Delgado, O. D. Matteo, A. Dusko, T. Garg, D. Guala, A. Hayes, R. Hill, A. Ijaz, T. Isaacsson, D. Ittah, S. Jahangiri, P. Jain, E. Jiang, A. Khandelwal, K. Kottmann, R. A. Lang, C. Lee, T. Loke, A. Lowe, K. McKiernan, J. J. Meyer, J. A. Montañez-Barrera, R. Moyard, Z. Niu, L. J. O’Riordan, S. Oud, A. Panigrahi, C.-Y. Park, D. Polatajko, N. Quesada, C. Roberts, N. Sá, I. Schoch, B. Shi, S. Shu, S. Sim, A. Singh, I. Strandberg, J. Soni, A. Száva, S. Thabet, R. A. Vargas-Hernández, T. Vincent, N. Vitucci, M. Weber, D. Wierichs, R. Wiersema, M. Willmann, V. Wong, S. Zhang, and N. Killoran, *Penylane: Automatic differentiation of hybrid quantum-classical computations* (2022), [arXiv:1811.04968](https://arxiv.org/abs/1811.04968) [quant-ph].
 - [5] Y. Suzuki, Y. Kawase, Y. Masumura, Y. Hiraga, M. Nakadai, J. Chen, K. M. Nakanishi, K. Mitarai, R. Imai, S. Tamiya, T. Yamamoto, T. Yan, T. Kawakubo, Y. O. Nakagawa, Y. Ibe, Y. Zhang, H. Yamashita, H. Yoshimura, A. Hayashi, and K. Fujii, *Quantum* **5**, 559 (2021).
 - [6] S. Efthymiou, S. Ramos-Calderer, C. Bravo-Prieto, A. Pérez-Salinas, D. García-Martín, A. García-Saez, J. I. Latorre, and S. Carrazza, *Quantum Science and Technology* **7**, 015018 (2021).
 - [7] S. Efthymiou, M. Lazzarin, A. Pasquale, and S. Carrazza, *Quantum* **6**, 814 (2022).
 - [8] R. Orús, *Annals of Physics* **349**, 117–158 (2014).
 - [9] J. Gray, *Journal of Open Source Software* **3**, 819 (2018).
 - [10] M. Fishman, S. R. White, and E. M. Stoudenmire, *SciPost Phys. Codebases*, 4 (2022).
 - [11] J. Hauschild and F. Pollmann, *SciPost Physics Lecture Notes* 10.21468/scipostphyslectnotes.5 (2018).
 - [12] M. Ballarin, F. P. Barone, A. Coppi, D. Jaschke, S. Montanero, G. M. Menés, D. Rattacaso, and N. Reinić, *Quantum tea: qmatchatea* (2025), cite for qmatchatea.
 - [13] S. Aaronson and D. Gottesman, *Physical Review A* **70**, 10.1103/physreva.70.052328 (2004).
 - [14] C. Gidney, *Quantum* **5**, 497 (2021).
 - [15] M. S. Rudolph, T. Jones, Y. Teng, A. Angrisani, and Z. Holmes, *Pauli propagation: A computational framework for simulating quantum systems* (2025), [arXiv:2505.21606](https://arxiv.org/abs/2505.21606) [quant-ph].
 - [16] M. L. Goh, M. Larocca, L. Cincio, M. Cerezo, and F. Sauvage, *Physical Review Research* **7**, 10.1103/3y65-f5w6 (2025).
 - [17] H. Lange, A. V. de Walle, A. Abedinnia, and A. Bohrdt, *From architectures to applications: A review of neural quantum states* (2024), [arXiv:2402.09402](https://arxiv.org/abs/2402.09402) [cond-mat.dis-nn].
 - [18] C. Gidney, *Quantum* **5**, 497 (2021).
 - [19] T. Begušić *et al.*, *Science Advances* **10**, eadk4321 (2024).
 - [20] T. Haug and L. Piroli, *Physical Review B* **107**, 10.1103/physrevb.107.035148 (2023).
 - [21] Q. addons team, *Pauli prop*, <https://github.com/Qiskit/pauli-prop> (2025).
 - [22] M. Robbiati, G. Crognaletti, and M. Robbiano, *MPStab docs: qibo backend provider* (2024), accessed: 2026-05-06.
 - [23] M. Robbiati, A. Papaluca, A. Pasquale, E. Pedicillo, R. M. S. Farias, A. Sopena, M. Robbiano, G. Alramahi, S. Bordoni, A. Candido, N. Laurora, J. S. Neto, Y. P. Tan, M. Grossi, and S. Carrazza, *Qiboml: towards the orchestration of quantum-classical machine learning* (2025), [arXiv:2510.11773](https://arxiv.org/abs/2510.11773) [quant-ph].
 - [24] R. Jozsa, *On the simulation of quantum circuits* (2006), [arXiv:quant-ph/0603163](https://arxiv.org/abs/quant-ph/0603163) [quant-ph].